Презентация по лабораторной работе №4

Компьютерный практикум по статистическому анализу

Арбатова Варвара Петровна

2026-10-13

Содержание I

1. Информация
2. Вводная часть
3. Теоретическая часть
4. Выполнение работы
5. Результаты
6. Обсуждение
7. Заключение

Раздел 1

Информация

Докладчик

- Арбатова Варвара Петровна
- Студентка
- Российский университет дружбы народов им. П. Лумумбы
- 1132236020@rudn.ru
- https://vparbatova.github.io/ru/

Раздел 2

Вводная часть

Актуальность

- Линейная алгебра — основа численных методов и анализа данных
- Julia предоставляет эффективные средства для работы с матрицами
- Важно уметь быстро и корректно решать типовые задачи
- Необходимо оценивать производительность операций

Объект и предмет исследования

- **Объект:** объекты линейной алгебры (векторы, матрицы) и операции над ними
- **Предмет:** эффективность и корректность реализации операций в пакетах Julia
- Рассматриваются: `LinearAlgebra`, `Statistics`, `BenchmarkTools`

Цели и задачи

- **Цель:**
  - Изучить возможности пакетов Julia для операций линейной алгебры и оценки их эффективности.
- **Задачи:**
  - Освоить поэлементные операции над массивами.
  - Изучить факторизации (LU, QR) и спектральное разложение.
  - Решить системы линейных уравнений.
  - Проверить продуктивность матриц тремя критериями.
  - Оценить производительность операций.

Материалы и методы

- **ПО:** Julia 1.12.7, пакеты `LinearAlgebra`, `Statistics`, `BenchmarkTools`
- **Среда:** Jupyter Notebook
- **Методы:**
  - Численные методы линейной алгебры
  - Факторизации: LU, QR, спектральное разложение
  - Бенчмаркинг через `@btime`

Раздел 3

Теоретическая часть

Позлементные операции

- `sum(a, dims=1)` — сумма по столбцам
- `sum(a, dims=2)` — сумма по строкам
- `prod`, `mean` — аналогично
- Пакет `Statistics` даёт `mean`

```
# Element-wise operations on a 4x3 array
a = rand(1:20, (4, 3))
sum(a)              # Total sum
sum(a, dims=1)      # Sum by columns
sum(a, dims=2)      # Sum by rows

using Statistics
mean(a)             # Average value
mean(a, dims=1)     # Mean by columns
```

Факторизации

- **LU:** $A = LU$, L — нижнетреугольная, U — верхнетреугольная
- **LUP:** $PA = LU$, P — матрица перестановок
- **QR:** $A = QR$, Q — ортогональная, R — верхнетреугольная
- **Спектральное:** $A = V\Lambda V^{-1}$

```
using LinearAlgebra
A = rand(3, 3)

Alu = lu(A)          # LU factorization
Aqr = qr(A)          # QR factorization
E   = eigen(A)       # Spectral decomposition

det(A); det(Alu)     # Determinant: direct and via factorization
Aqr.Q' * Aqr.Q       # Check: Q is orthogonal
```

Продуктивность матриц

- Матрица A **продуктивна**, если решение x системы $x - Ax = y$ при любом $y \geq 0$ даёт $x \geq 0$
- **Три эквивалентных критерия:**
  1. Решение по определению
  2. Все элементы $(E - A)^{-1} \geq 0$
  3. Все собственные значения $|\lambda| < 1$

Раздел 4

Выполнение работы

Задание 1. Произведение векторов

```
# Task 1: Vector products
v = [1, 2, 3]

dot_v = dot(v, v)       # Scalar product
outer_v = v * v'        # Outer product
```

Результат:

```
dot_v = 14
outer_v =
 1   2   3
 2   4   6
 3   6   9
```

Задание 2. Системы линейных уравнений (2 неизвестных)

```julia
# Task 2: Systems of linear equations (2 unknowns)
using LinearAlgebra

# a) x + y = 2; x - y = 3
A = [1 1; 1 -1]; b = [2, 3]
x = A \ b
println("1a: x = ", x)

# c) x + y = 2; 2x + 2y = 5 -> inconsistent
A = [1 1; 2 2]; b = [2, 5]
println("1c: rank(A) = ", rank(A), ", rank([A b]) = ", rank([A b]))
```

Результат:

```
1a: x = [2.5, -0.5]
1c: rank(A) = 1, rank([A b]) = 2
```

Задание 2. Системы линейных уравнений (3 неизвестных)

```
# Task 2: Systems with 3 unknowns
A = [1 1 1; 2 2 -3; 3 1 1]
b = [2, 4, 1]
println("2b: det(A) = ", det(A))
x = A \ b
println("2b: x = ", x)

# Underdetermined system: use pseudoinverse
A = [1 1 1; 1 -1 -2]; b = [2, 3]
x = pinv(A) * b
println("2a: x = ", x)
```

Результат:

```
2b: det(A) = -10.0
2b: x = [-0.5, 2.5, 0.0]
```

Задание 3. Приведение матриц к диагональному виду

```
# Task 3: Diagonalization
A = [1 -2; -2 1]
E = eigen(A)

V = E.vectors
Lambda = Diagonal(E.values)
A_diag = V * Lambda * inv(V)

println("Eigenvalues: ", E.values)
```

Результат:

```
1a: eigenvalues = [-1.0, 3.0]
1b: eigenvalues = [2.0, 4.0]
1c: eigenvalues = [-1.0, 3.0]
```

Задание 3. Вычисление степеней и корней матриц

```
# Task 3: Matrix powers and roots
A = [1 -2; -2 1]
println("2a: A^10 = ", A^10)

B = [5 -2; -2 5]
println("2b: sqrt(B) = ", sqrt(B))

C = [1 2; 2 3]
println("2d: sqrt(C) = ", sqrt(C))
```

Результат:

```
2a: A^10 = [29525 -29524; -29524 29525]
2b: sqrt(B) = [2.1889 -0.4569; -0.4569 2.1889]
2d: sqrt(C) is complex-valued
```

Задание 3. Собственные значения матрицы 5×5

```
# Task 3: Eigenvalues of a 5x5 symmetric matrix
A = [140  97  74 168 131;
      97 106  89 131  36;
      74  89 152 144  71;
     168 131 144  54 142;
     131  36  71 142  36]

E = eigen(A)
println("Eigenvalues: ", E.values)
println("Symmetric? ", issymmetric(A))
```

Результат:

```
Eigenvalues: [-128.493, -55.888, 42.752, 87.161, 542.468]
Symmetric? true
```

Задание 4. Продуктивность матриц

```
# Task 4: Productivity check (3 criteria)
using LinearAlgebra

function is_productive_spec(A)
    lambda = eigvals(A)
    max_abs = maximum(abs.(lambda))
    return max_abs < 1, lambda, max_abs
end

A1 = [1 2; 3 1]
ok, lambda, m = is_productive_spec(A1)
println("3a: max|lambda| = ", m, " -> productive? ", ok)
```

Результат:

```
3a: max|lambda| = 3.449  -> productive? false
```

Раздел 5

Результаты

Эффективность операций (BenchmarkTools)

```
# Performance evaluation
using BenchmarkTools
n = 1000
A = randn(n, n)
Asym = A + A'

@btime eigvals(Asym);
@btime eigvals(Asym_noisy);
@btime eigvals(Asym_explicit);
```

Результат:

```
150.6 ms  (18 allocations:  7.98 MiB)   # eigvals(Asym)
  1.42 s  (27 allocations:  7.93 MiB)   # eigvals(Asym_noisy)
148.2 ms  (18 allocations:  7.98 MiB)   # eigvals(Asym_explicit)
 1.12 s  (39 allocations: 183.11 MiB)   # eigmax(SymTridiagonal)
```

Основные результаты

- Освоены базовые операции Julia для работы с массивами и матрицами
- Получены навыки работы с LU-, QR- и спектральным разложением
- Решены 6 СЛАУ с двумя неизвестными и 4 СЛАУ с тремя неизвестными
- Найдены собственные значения матрицы 5×5
- Проверена продуктивность матриц тремя критериями — результаты согласованы
- Оценена производительность операций через `BenchmarkTools`

Раздел 6

Обсуждение

Анализ результатов

- **Согласованность критериев продуктивности:** все три критерия дали одинаковые результаты
- **Производительность:**
  - `Symmetric` ускоряет `eigvals` в ~10 раз
  - `SymTridiagonal` позволяет работать с матрицами $10^6 \times 10^6$
- **Комплексные результаты:** `sqrt` и `A^(1/3)` для неотрицательно определённых матриц могут давать комплексные значения
- **Несовместные системы:** выявлены через сравнение рангов

Раздел 7

Заключение

Главный вывод

Использование Julia и её пакетов позволяет эффективно решать задачи линейной алгебры, а явное указание структуры матриц даёт значительный прирост производительности.